

Metodologie di Programmazione (A-L)

Il semestre - a.a. 2025 – 2026

Array

a cura di Massimo La Morgia



SAPIENZA
UNIVERSITÀ DI ROMA

*Tutti i diritti relativi al presente materiale didattico ed al suo contenuto sono riservati a Sapienza e ai suoi autori (o docenti che lo hanno prodotto). È consentito l'uso personale dello stesso da parte dello studente a fini di studio. Ne è vietata nel modo più assoluto la diffusione, duplicazione, cessione, trasmissione, distribuzione a terzi o al pubblico pena le sanzioni applicabili per legge.

**I crediti sulle slide di questo corso sono riportati nell'ultima slide

Array

Array

- Un **array** rappresenta un **gruppo di variabili** (chiamate **elementi**) tutte dello **stesso tipo**
- **Gli array sono oggetti**
 - Quindi le variabili di tipo array contengono il **riferimento all'array**
- Gli **elementi** di un array possono essere **tipi primitivi** (interi, double, ecc.) oppure **riferimenti a oggetti** (inclusi altri array!)

```
public static void main(String[] args) {  
    int[] arrayOfInteger = new int[10];  
    System.out.println(arrayOfInteger);  
}
```



[I@79fc0f2f]

Un esempio di array: creazione senza valori

- Dichiarazione: `int[] a;`

Nome dell'array

- Creazione senza valori:

```
a = new int[10];
```

a 0x00AB0125

Riferimento all'array

Identificatore del primo elemento

a[0]
a[1]
a[2]
a[3]
a[4]
a[5]
a[6]
a[7]
a[8]
a[9]

0
0
0
0
0
0
0
0
0
0

Valore del primo elemento

Ogni elemento viene
inizializzato con il valore di
default (0, false, null)

Un esempio di array: creazione con valori

- Dichiarazione: `int[] a;`

Nome dell'array

- Creazione con valori:

```
a = new int[] { 5, -2, 3, 0, 1, -6, 75, 32, 122, 4 };
```

a 0x00AB0125

Riferimento all'array

Identificatore del primo elemento

a[0]	5
a[1]	-2
a[2]	3
a[3]	0
a[4]	1
a[5]	-6
a[6]	75
a[7]	32
a[8]	122
a[9]	4

Valore del primo elemento

Ogni elemento viene inizializzato con il valore specificato

Dichiarazioni valide di array

Un array di 10 interi. Tutti
inizializzati a 0.

```
int[] numeri = new int[10];
```

Meglio utilizzare una
COSTANTE per la
dimensione di un array.

```
final int NUMERO_DI_CIFRE = 10;  
int[] numeri = new int[NUMERO_DI_CIFRE];
```

La lunghezza di un array può
essere specificata con una
variabile

```
int numeroDiCifre = new Scanner(System.in).nextInt();  
int[] numeri = new int[numeroDiCifre];
```

Un array di 10 interi
costruito specificando i
valori

```
int[] numeri = { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 };
```

Un array di 3 riferimenti a
stringa, tutti inizializzati a
null

```
String[] nomi = new String[3];
```

Un array di 3 riferimenti a
stringa, con valori
assegnati

```
String[] nomi = { "mario", "luigi", "wario" };
```

Errori tipici con gli array

- Al contrario del linguaggio C, in Java non è possibile specificare la dimensione accanto al nome dell'array:

```
int a[10];
```

- Né specificare la dimensione e allo stesso tempo inizializzare i valori:

```
int[] a = new int[10] { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
```

Accedere agli elementi di un array

- Si **accede** a un elemento dell'array specificando il **nome dell'array** seguito dalla **posizione (indice)** dell'elemento tra **parentesi quadre**

```
// stampa il sesto elemento dell'array  
System.out.println(a[5]);
```

```
// memorizza la somma dei primi 3 elementi  
int k = a[0] + a[1] + a[2];
```

- L'indice è **sempre positivo**, compreso tra 0 e la dimensione dell'array-1
- L'indice può essere un'**espressione**:

```
int i = 2;  
int j = 3;  
  
a[i+j*2] += 2;
```

Esercizio: stampa di un array

- Scrivere un metodo che, dato un array di stringhe, ne stampi i valori in sequenza
- Ad esempio, dato l'array { "son", "buoni", "almeno?", "assaggi", "il", "vino" } stampi: ["son", "buoni", "almeno?", "assaggi", " il", "vino"]

```
public void stampaArray(String[] a)
{
    System.out.print("[ ");

    for (int k = 0; k < a.length; k++)
        System.out.print "\"" + a[k] + "\""
            + (k == a.length-1 ? " ]" : ", ");

    System.out.println();
}
```

Lunghezza dell'array
(**attenzione: .length
SENZA PARENTESI**)

Esercizio: somma dei valori di un array

- Scrivere un metodo che, dato un array di interi, restituisca la somma dei suoi elementi
- Ad esempio, dato l'array { 10, 12, 12, 8 } il metodo deve restituire: 42

```
public int sommaArray(int[] a)
{
    int val = 0;

    for (int k = 0; k < a.length; k++) val += a[k];

    return val;
}
```

Esercizio: media dei valori di un array

- Utilizzando il metodo scritto per l'esercizio precedente, scrivere un metodo che, dato un array di interi, **restituisca la media (double) dei suoi elementi**
- Ad esempio, dato l'array { 10, 12, 12, 8 } il metodo deve restituire: **10.5**

```
public double mediaArray(int[] a)
{
    return sommaArray(a)/a.length;
}
```

occhio: divide intero per intero (cosa serve?)

RIUSO DEL CODICE!!!

- Scrivete anche la versione per **array di double!**

Altri esercizi molto semplici con gli array

- Scrivere un metodo che, dato un array di stringhe e una stringa in input, restituisca **true** se l'array contiene la stringa, **false** altrimenti
- Scrivere una seconda versione del metodo che restituisca la **posizione della stringa trovata**, -1 altrimenti
- Scrivere un metodo che, dato un array di double, restituisca il **valore massimo dell'array**

Esercizio: reimplementa conta vocali

- Scrivere un metodo che riceve una stringa e stampa a video il conteggio delle vocali in essa contenute
 - Utilizzare un array per il conteggio separato delle 5 vocali
- Ad esempio: data la stringa "le aiuole sono pulite", il metodo stampa: a=1 e=3 i=2 o=3 u=2
- Come l'avete implementato **SENZA** array?

Esercizio: array con fattoriale

- Scrivere un metodo che, dato un intero n in ingresso, restituisca un array di dimensione n contenente $k!$ nella k -esima cella, per ogni valore di k

```
public int[] getArrayFattoriali(final int n)
{
    int[] fatt = new int[n];

    fatt[0] = 1;
    for (int k = 1; k < n; k++) fatt[k] = fatt[k-1]*k;

    return fatt;
}
```

Esercizio: "l'array risponde"

Progettare una classe `MioArray` i cui oggetti vengono costruiti con un array di interi (`int[ ]`)

- La classe implementa i seguenti metodi:
 - `contiene`, che dati in ingresso una posizione e un intero, restituisce `true` o `false` se l'intero è contenuto in quella posizione nell'array
 - `somma2`, che restituisce la somma dei primi due elementi dell'array. Se l'array è di lunghezza inferiore (**info**: la lunghezza dell'array si ottiene con il campo speciale `length`, quindi `a.length`), restituisce il valore del primo elemento oppure 0 se l'array è vuoto
 - `scambia`, che date in ingresso due posizioni intere, scambia i valori presenti nelle due posizioni dell'array (es. `scambia(1, 3)` trasforma l'array { 1, 2, 3, 4, 5 } in { 1, 4, 3, 2, 5 })
 - `maxTripla`: che restituisce il valore massimo tra il primo, l'ultimo e il valore in posizione intermedia dell'array (es. restituisce 3 se l'oggetto è costruito con { **1**, 7, 5, **3**, 0, 2, **2** }, le posizioni esaminate sono in grassetto)
 - `falloInDue`: che restituisce un array di due interi, il primo è il primo elemento dell'array dell'oggetto, il secondo è l'ultimo elemento dell'array dell'oggetto

Esercizio: stampa di istogrammi

- Progettare una classe `Istogramma` che rappresenta la distribuzione di dati (es. voti degli studenti) in un intervallo da `i` a `j` fornito in input (es. da `0` a `31` (trenta e lode))
- La classe permette di incrementare il conteggio in corrispondenza di ciascun elemento dell'intervallo (es. memorizzando così un nuovo voto di uno studente)
- La classe può stampare a video l'istogramma corrispondente
 - Più facile in orizzontale
 - Provate a stampare in verticale!!!

Esercizio: mescolare e distribuire un mazzo di carte da gioco

- Progettare una classe `Carta` che rappresenti una singola carta da gioco (con seme e valore)
 - La classe deve restituire su richiesta la propria rappresentazione sotto forma di stringa
- Progettare quindi una classe `MazzoDiCarte` che rappresenti un intero mazzo da `52 carte`
- La classe deve `implementare` i seguenti metodi:
 - `mescola` il mazzo di carte
 - `distribuisci` la prossima carta
- Infine si progetti una `classe di collaudo` che crea un mazzo, mescoli le carte e ne distribuisca carte fino ad esaurimento del mazzo

Esercizio: DaCifreALettere

- Scrivere un metodo che prenda in ingresso una stringa contenente cifre e **restituisca una stringa in cui ciascuna cifra è stata trasformata nella parola corrispondente**
- Ad esempio, data in input la stringa "8452", il metodo restituisce "otto quattro cinque due"
- Viceversa, scrivere un metodo che prenda in ingresso una stringa contenente cifre scritte a lettere e **restituisca una stringa contenente le cifre corrispondenti**
- Ad esempio, data in input la stringa "otto quattro cinque due", il metodo restituisce "8452"
- **Nota:** è conveniente utilizzare gli array di stringhe `String[]`!

Esercizio: da cifre a lettere e viceversa (avanzato!!!)

- Come l'esercizio precedente, ma stampando (o leggendo) a lettere tenendo conto della posizione delle cifre (occhio ai **casi speciali**: undici, dodici, ecc...)
- Ad esempio, "8452" viene trasformato in "ottomila quattrocento cinquanta due"

Modificare le dimensioni di un array

- Un array ha **dimensioni prefissate** che **NON** possono essere modificate
- Tuttavia è possibile creare un **nuovo array con nuove dimensioni** a partire da un array preesistente

Con il **metodo statico copyOf** della classe `java.util.Arrays`

```
import java.util.Arrays;
```

```
public class MyArrays  
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        // array di dimensione 9
```

```
        int[] array = { 1, 5, 8, 2, 3, 4, 7, 6, 9 };
```

```
        System.out.println(Arrays.toString(array));
```

```
        // restringe l'array a dimensione 5
```

```
        array = Arrays.copyOf(array, 5);
```

```
        System.out.println(Arrays.toString(array));
```

```
        // allarga l'array a dimensione 8 (gli ultimi 3 valori sono inizializzati a 0)
```

```
        array = Arrays.copyOf(array, 8);
```

```
        System.out.println(Arrays.toString(array));
```

```
    }
```

```
}
```

Crea un nuovo array
con i primi 5 elementi
dell' array in input

Crea un nuovo array
di 8 elementi (i primi
elementi dall' array in
input)

Metodo per la
formattazione a
stringa di un array

Output

[1, 5, 8, 2, 3, 4, 7, 6, 9]

[1, 5, 8, 2, 3]

[1, 5, 8, 2, 3, 0, 0, 0]

Esercizio: implementare un filtro

- Progettare una classe `Filtro` costruita con un array di interi
- La classe implementa operazioni che permettono di ottenere nuovi sotto-array dell'array iniziale:
 - `passaBasso`: restituisce tutti gli elementi $\leq k$ nell'ordine iniziale
 - `passaAlto`: restituisce tutti gli elementi $\geq k$ nell'ordine iniziale
 - `filtra`: restituisce l'array iniziale da cui sono state eliminate tutte le occorrenze dell'intero passato in input
 - `filtra`: una seconda versione del metodo che restituisce l'array iniziale da cui vengono eliminate tutte le occorrenze di interi presenti nell'array passato in input
- Se `Filtro` viene costruito con l'array `{ 1, 2, 10, 2, 42, 7, 8 }`:
 - `passaBasso(8)` restituisce `{ 1, 2, 2, 7, 8 }`
 - `passaAlto(9)` restituisce `{ 10, 42 }`
 - `filtra(2)` restituisce `{ 1, 10, 42, 7, 8 }`
 - `filtra(new int[] { 2, 7, 42 })` restituisce `{ 1, 10, 8 }`

Esercizio: reimplementare `Arrays.copyOf()` e `Arrays.toString()`

- Implementare un metodo statico `copyOf` che, analogamente a `java.util.Arrays.copyOf`, copi un array in un nuovo array delle dimensioni specificate (`troncando` l'array in input, se più grande)

Esercizio: sequenza di cifre estensibile

Progettare una classe `SequenzaDiCifre` che espone un metodo che, data in input una stringa e un intero N, aggiunga alla sequenza inizialmente vuota (rappresentata mediante un array) le prime N cifre contenute nella stringa (si assuma che ne contenga comunque almeno N). La classe espone anche un metodo `toString` che fornisce una rappresentazione sotto forma di stringa della sequenza.

Ad esempio:

- `SequenzaDiCifre s = new SequenzaDiCifre();`
- `s.aggiungiCifre("abc1--23", 2);`
- `s.aggiungiCifre("xx0a8b76543100", 4);`
- `System.out.println(s.toString());`

stampa: `[1,2,0,8,7,6]`

Esercizio: implementare una lista mediante array

- Che cos'è una lista? E' una sequenza di oggetti
- Implementare una classe `ListaDiInteri` che permetta le seguenti operazioni:
 - `Restituisce` l'elemento i-esimo della lista
 - `Restituisce` l'indice della posizione dell'intero fornito in input
 - `Restituisce una stringa formattata` contenente la lista di interi
 - `Restituisce` la dimensione della lista
 - `Contiene` un determinato intero (true o false)?
 - `Aggiungi` un intero in coda alla lista
 - `Aggiungi` un intero nella posizione specificata
 - `Elimina` la prima occorrenza di un intero dalla lista
 - `Elimina` l'elemento i-esimo della lista

Metodi con un numero di parametri variabile (Java >=5)

- Si possono dichiarare metodi con un numero variabile di parametri (a partire da Java 5)
- Mediante la sintassi: **tipo...**

```
public class SommaDouble
{
    public static double sum(double... valori)
    {
        double somma = 0.0;
        for (int k = 0; k < valori.length; k++) somma += valori[k];
        return somma;
    }

    public static double sumFirstN(final int N, double... valori)
    {
        double somma = 0.0;
        for (int k = 0; k < valori.length && k < N; k++) somma += valori[k];
        return somma;
    }

    public static void main(String[] args)
    {
        System.out.println(sum());
        System.out.println(sum(1, 2, 3, 4));
        System.out.println(sumFirstN(3, 1, 2, 3, 4));
    }
}
```

Zero, uno o più
valori double

Di fatto è un
riferimento ad array

E' possibile
specificare altri
parametri, ma
PRIMA dell'**UNICA**
sequenza variabile
di parametri

Output:

0
10
6

Array a 2 dimensioni

- Possiamo specificare un **array a 2 dimensioni** (o **matrice**) semplicemente specificando due coppie di parentesi quadre []:

```
String[][] matrice = new String[RIGHE][COLONNE];
```

- L'accesso avviene specificando le due dimensioni dell'array:

```
System.out.println(matrice[y][x]);
```

Esercizio: la tavola pitagorica

- Scrivere una classe che rappresenti la tavola pitagorica $N \times N$ (dove l'intero N è un parametro di costruzione della classe)
- La classe deve, su richiesta, restituire il valore della tabella in corrispondenza della posizione (i, j)
- La classe deve poter stampare l'intera tavola

x	0	1	2	3	4	5	6	7	8	9	10
0	0	0	0	0	0	0	0	0	0	0	0
1	0	1	2	3	4	5	6	7	8	9	10
2	0	2	4	6	8	10	12	14	16	18	20
3	0	3	6	9	12	15	18	21	24	27	30
4	0	4	8	12	16	20	24	28	32	36	40
5	0	5	10	15	20	25	30	35	40	45	50
6	0	6	12	18	24	30	36	42	48	54	60
7	0	7	14	21	28	35	42	49	56	63	70
8	0	8	16	24	32	40	48	56	64	72	80
9	0	9	18	27	36	45	54	63	72	81	90
10	0	10	20	30	40	50	60	70	80	90	100

Esercizio: il gioco del tris

- Progettare una classe `ScacchieraTris` che implementi la scacchiera del gioco del tris
- La classe deve **memorizzare la scacchiera** i cui elementi possono essere:
 - " " (se non è stata ancora occupata la casella)
 - "X" oppure "O" (secondo il giocatore che ha occupato la casella)
- La classe deve stampare in qualsiasi momento la **situazione della scacchiera**
- Deve permettere di occupare una casella con un simbolo "X" o "O"
- Progettare quindi una classe `Tris` che implementi il gioco utilizzando la scacchiera appena progettata

Esercizio: calcolo di media, mediana e moda

- Progettare una classe `Sequenza` i cui oggetti si costruiscono con un array di interi
- La classe espone i seguenti metodi:
 - `getMediana()` che restituisce l'elemento centrale dell'array ordinato (si utilizzi `Arrays.sort` per ordinare l'array e `Arrays.copyOf` per effettuarne una copia)
 - `getMedia()` che restituisce la media degli elementi dell'array
 - `getModa()` che restituisce il valore più frequente nella sequenza

Credits

Le slide di questo corso sono basate sulle slide del Prof. Faralli che sono a loro volta una rielaborazione delle slide del Prof. Navigli

In aggiunta, le slide sono state revisionate dagli studenti borsisti della Facoltà di Ingegneria Informatica, Informatica e Statistica: Mario Marra e Paolo Straniero.

Font ad alta leggibilità Biancoenero® di biancoenero edizioni srl, disegnata da Umberto Mischi, con la consulenza di Alessandra Finzi, Daniele Zanoni e Luciano Perondi (Brevetto n. RM20110000128).

Disponibile gratuitamente per tutte le istituzioni e i privati che la utilizzino per scopi non commerciali.